

EventForce Management System

Salesforce Implementation | Project Documentation

Client: Event Planner Solutions

Platform: Salesforce Developer Edition Org (Lightning Experience)

Stack: Custom Objects, Flows, Approval Process, Apex, Reports & Dashboards, Security Model

Phases: Requirements > Backend > UI/UX > Data, Testing & Security > Deployment & Maintenance

Table of Contents

- 1. Project Overview & Objectives 3**
- 2. Phase 2: Backend Development & Configuration 4**
 - Milestone 1: Salesforce Developer Account 4
 - Data Architecture 4
 - Milestone 2: Custom Objects 5
 - Milestone 3: Custom Tabs 5
 - Milestone 4: Fields & Relationships 6
 - Milestone 5: Validation Rules 9
 - Milestone 6: Approval Process (Event Cancellation) 9
 - Milestone 7: Flow - 3-Day Reminder Email 13
 - Milestone 8: Apex Class & Trigger - Venue Status 14
 - Milestone 9: Apex Trigger - Prevent Double Booking 15
 - Milestone 10: Batch & Schedulable Apex 16
- 3. Phase 3: UI/UX Development & Customization 18**
 - Milestone 11: Lightning App 18
 - Milestone 12: Reports 19
 - Milestone 13: Dashboard 20
- 4. Phase 4: Data Migration, Testing & Security 20**
 - Milestone 14: Profiles 20
 - Milestone 15: Roles 21
 - Milestone 16: Users 22
 - Milestone 17: Permission Set 23
 - Milestone 18: Sharing Settings 23
- 5. Phase 5: Deployment, Documentation & Maintenance 25**

1. Project Overview & Objectives

Business name: EventForce Management System (client: Event Planner Solutions)

EventForce is a Salesforce CRM solution that digitizes and automates event-management operations: client bookings, venue reservations, vendor coordination and feedback collection, with real-time visibility through reports and dashboards. Key processes such as event scheduling, cancellation approvals and client reminders are automated.

Objectives

- Centralize client, event, vendor, venue and feedback data in one CRM platform.
- Automate workflows (reminders, approvals, venue status) to **reduce manual processes by 60%**.
- Improve booking efficiency by **40%** and client satisfaction through timely communication.
- Provide data-driven insight through integrated reporting and analytics.
- Improve collaboration between event coordinators, vendor managers and clients.

Phase 1: Requirement Analysis & Planning

Functional scope

- **Custom objects:** Event, Client, Vendor, Venue, Feedback and Event Vendor (junction object).
- **Relationships:** Lookup, Master-Detail and Many-to-Many (Event ↔ Vendor).
- **Flows** for reminders, feedback notifications and approval updates.
- **Validation rules and formula fields** for accurate data entry and event budget calculation.
- **Approval process** for event cancellation, with email alerts to coordinators and clients.
- **Profiles, roles and permission sets** for user-level access and security.
- **Reports and dashboards** for event summary, vendor performance and upcoming-event insights.

Stakeholder mapping

Stakeholder	Responsibility
Clients	Request event services, provide feedback, view event updates.
Event Coordinators	Manage event scheduling, client communication and approval workflows.
Vendors	Handle assigned services such as catering, decor and logistics.
Venue Managers	Manage venue availability and reservations.
Event Administrators	Oversee system operations, security and data monitoring.

Execution roadmap

- 1 Object & field setup:** Event, Client, Vendor, Venue, Feedback, Event Vendor.
- 2 Automation flows:** record-triggered flows for reminders and approvals.
- 3 Validation rules & formula fields:** data accuracy and budget calculation.
- 4 Profiles, roles, permission sets:** user access and record visibility.
- 5 Approval process:** event-cancellation workflow with notifications.
- 6 Apex classes & triggers:** venue availability update and double-booking prevention.
- 7 Reports & dashboards:** analytics for upcoming events and performance.

2. Phase 2: Backend Development & Configuration

This phase built the core engine of the system using declarative tools (record-triggered flows, validation rules, formula fields, approval processes) together with Apex classes and triggers for advanced logic such as venue availability updates and double-booking prevention. Profiles, roles, permission sets and sharing rules secure data across all modules.

Milestone 1: Salesforce Developer Account

Salesforce is a customer success platform for selling, servicing, marketing, analyzing and connecting with customers, with data stored securely in the cloud.

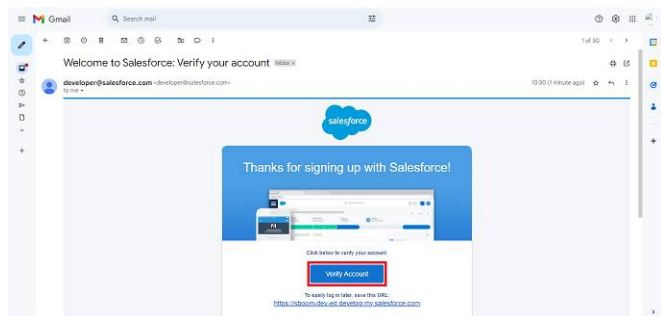
Activity 1: Create the Developer Org

- 1 Go to **developer.salesforce.com/signup**.
- 2 Enter first and last name, email, Role = **Developer**, Company = college name, Country = **India**, postal code, and a unique username in the format *username@organization.com* (it need not be a real email).
- 3 Click **Sign me up**.

Activity 2: Activate the account

- 1 Open the verification email (it can take 5-10 minutes) and click **Verify Account**.
- 2 Set a password, answer the security question and click **Change Password**; you are redirected to Setup.

Developer Edition sign-up form



Verification email

Data Architecture

Object	API name	Fields
Event	Event__c	Event Name, Event Date, Event Type, Event Status, Event Budget
Client	Client__c	Client Name, Email, Phone, Address, Country, City
Vendor	Vendor__c	Vendor Name, Email, Phone, Service Type, Status
Venue	Venue__c	Venue Name, Address, Location (URL), Capacity, Availability Status
Feedback	Feedback__c	Feedback ID (Auto Number), Rating, Comments
Event Vendor (junction)	EventVendor__c	Links Event and Vendor (many-to-many)

Relationship	Type
Event → Client	Lookup

Relationship	Type
Event → Venue	Lookup
Feedback → Event	Lookup
Feedback → Client	Lookup
EventVendor → Event	Master-Detail
EventVendor → Vendor	Master-Detail

Milestone 2: Custom Objects

Salesforce objects are database tables that store organization-specific data; they consist of fields (columns) and records (rows). The application uses five custom objects plus one junction object.

Activity 1: Create the Event object

- 1 From Setup open **Object Manager** → **Create** → **Custom Object**.
- 2 Label: **Event**; Plural label: **Events**; Record Name: **Event Name**; data type **Text**.
- 3 Optional features: **Allow Reports, Allow Activities, Track Field History**.
- 4 Deployment Status: **Deployed**; Search Status: **Allow Search**.
- 5 Object creation options: **Add Notes and Attachments related list to default page layout**.
- 6 Leave everything else as is and click **Save**.

Activities 2-5: Client, Vendor, Venue and Feedback objects

Repeat the same path (gear icon → Setup → Object Manager → Create → Custom Object) with the settings below. In each case tick **Allow Reports, Allow Search** and **Add Notes and Attachments related list to default page layout**, then Save.

Object	Label / Plural	Record name	Data type
Client	Client / Clients	Client Name	Text
Vendor	Vendor / Vendors	Vendor Name	Text
Venue	Venue / Venues	Venue Name	Text
Feedback	Feedback / Feedbacks	Feedback	Auto Number (display format F-{0000} , starting number 0)

New Custom Object definition

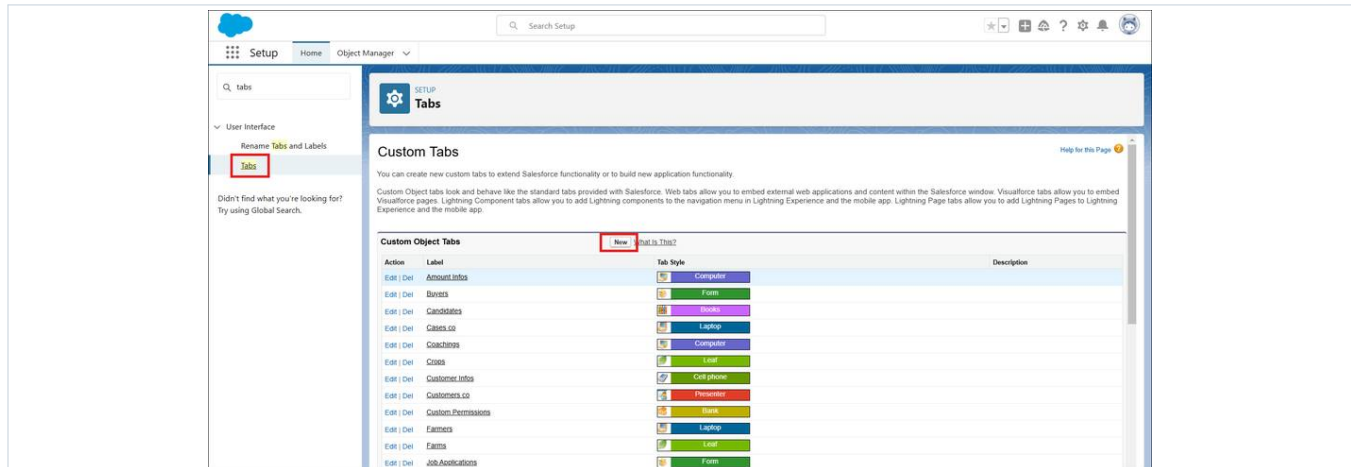
Optional features and creation options

Milestone 3: Custom Tabs

Tabs show the data of objects at a glance. Custom object tabs look and function like standard tabs.

- 1 On the Home tab type **Tabs** in Quick Find and select **Tabs**.
- 2 Under **Custom Object Tabs** click **New**.
- 3 Select the object (Event, Client, Vendor, Venue, then Feedback) and pick any Tab Style icon.

4 Leave defaults, click **Next, Next, Save**. Repeat for all five objects.



Custom tabs created for the five objects

Milestone 4: Fields & Relationships

Fields are the columns of an object. **Standard fields** (Created By, Last Modified By, Owner and the record-name field) cannot be deleted. **Custom fields** are added by the admin to meet business needs.

How to create a field

- 1 Gear icon → Setup → **Object Manager** → choose the object.
- 2 **Fields & Relationships** → **New**.
- 3 Choose the data type, enter the label, then **Next, Next, Save** (or **Save & New**). Tip: search first to make sure a similar field does not already exist.

Activity 1: Event fields

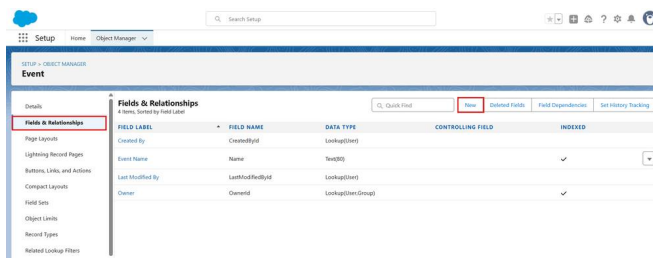
Field	Type	Values
Event Type	Picklist	Wedding, Corporate, Birthday, Anniversary, Festival, Concert, Other
Event Date	Date	-
Event Status	Picklist	Planned, Confirmed, Completed, Pending Cancellation, Canceled, Rejected
Event Budget	Formula (Currency)	Calculated from Event Type (below)

Event Budget formula (select return type Currency, check syntax, Next, Next, Save)

```

CASE(
  TEXT(Event_Type__c),
  "Wedding",    50000,
  "Corporate",  30000,
  "Birthday",   10000,
  "Anniversary", 20000,
  "Festival",   60000,
  "Concert",    40000,
  "Other",      15000,
  0
)

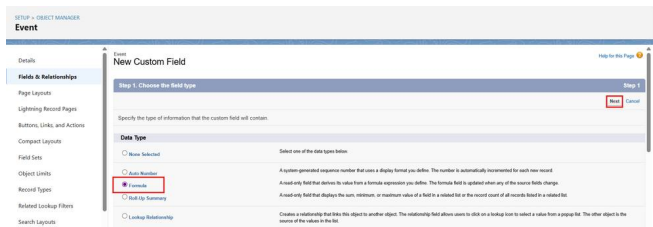
```



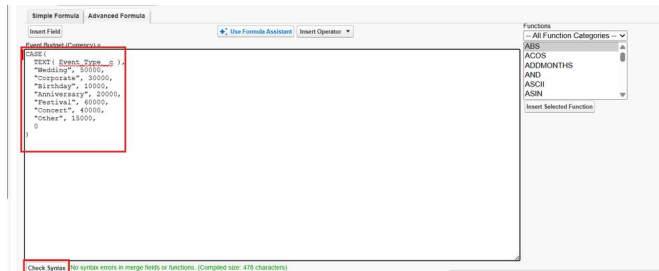
Fields & Relationships list



Picklist values for Event Type



Formula data type



Formula editor for Event Budget

Activity 2: Client fields

Field	Type	Values
Email	Email	-
Phone	Phone	-
Address	Text Area	-
Country	Picklist	India, Philippines, Saudi Arabia, USA, Switzerland, Thailand, Maldives, Dubai, Sri Lanka, Bangkok, Singapore, Myanmar, France, Nepal
City	Picklist	Hyderabad, Mumbai, Delhi, Bangalore, Chennai, Alaminos, Angeles City, Palayan, Quezon City, San Juan, Riyadh, Jeddah, Tayma, Dammam, Medina, New York, Los Angeles, Chicago, Houston, Columbus, Geneva, Basel, Lausanne, Bern, Lucerne

Activity 3: Vendor fields

Field	Type	Values
Service Type	Multi-Select Picklist	Catering, Decor, Photography, Videography, Lighting, Stage Setup, Makeup Artist, DJ/Music, Transportation, Hosting/Anchor
Email	Email	-
Phone	Phone	-
Status	Picklist	Available, Booked, Cancelled

Activity 4: Venue fields

Field	Type	Values
Address	Text Area	-
Location	URL	-
Capacity	Number	-
Availability Status	Picklist	Available, Reserved, Not Available

Activity 5: Feedback fields

Field	Type	Values
Rating	Picklist	1, 2, 3, 4, 5
Comments	Long Text Area	-

Relationships

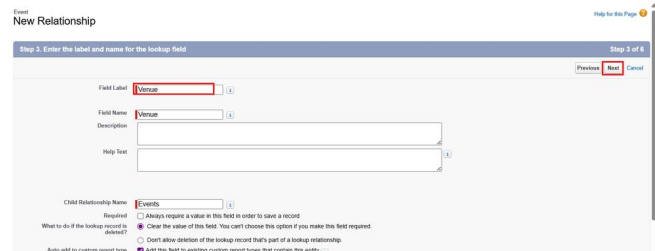
Lookups on Event: Object Manager → Event → Fields & Relationships → New → **Lookup Relationship** → related object **Venue** (label Venue) → Next, Next, Next, Save. Repeat with related object **Client** (label Client).

Many-to-Many (Event ↔ Vendor): Salesforce has no direct many-to-many, so create the junction object **Event Vendor** (*EventVendor__c*). On it, create two **Master-Detail** relationships: first to **Event** (label Event, Save & New), then to **Vendor** (label Vendor, Save).

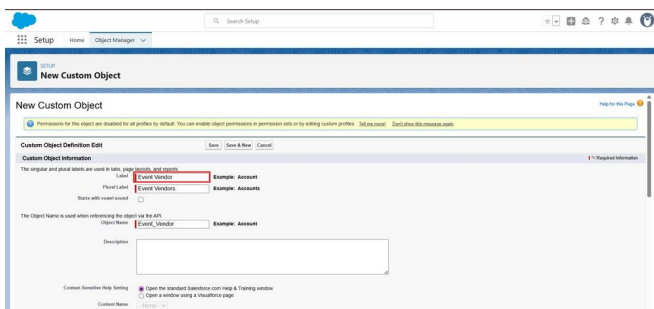
Lookups on Feedback: create a Lookup to **Event** and a Lookup to **Client** using the same steps.



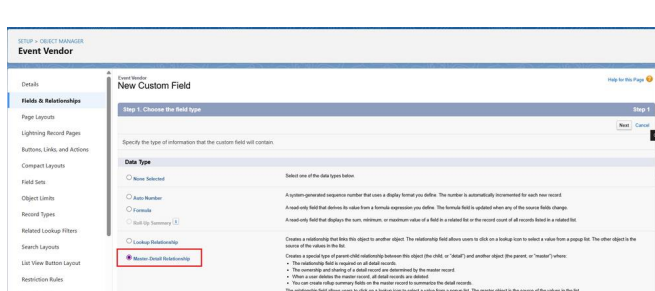
Creating the lookup relationship



Relationship settings



Creating the Event Vendor junction object



Master-Detail relationship

Activity 3: Lookup filter (Feedback → Event)

Scenario: every Event belongs to a Client, so when a Feedback is created and a Client is selected, the Event lookup should show only that Client's events.

- 1 Setup → Object Manager → **Feedback** → Fields & Relationships → open the **Event** lookup field (Event__c) → **Edit**.
- 2 Scroll to **Lookup Filter** and click **Show Filter Settings**.
- 3 Filter criteria: Field **Event: Client__c**, Operator **equals**, Value = **Field** radio → **Feedback: Client__c**.
- 4 Tick **Enable this filter** and click **Save**.

Lookup filter configuration

Milestone 5: Validation Rules

Validation rules run when a user saves a record. If the criteria are not met, an error message is shown and the record cannot be saved until the data is corrected.

- 1 Setup → Object Manager → **Client** → **Validation Rules** → **New**.
- 2 Rule name: **Email_Valid_Address**; tick **Active**.
- 3 Error condition formula (below).
- 4 Error message: **Please Enter Valid Email Address**; error location: **Field** → Email; click **Save**.

Error condition formula

```
NOT(REGEX(Email__c, "^[a-zA-Z0-9._]+@[a-zA-Z0-9.]+\.[a-zA-Z]{2,}$"))
```

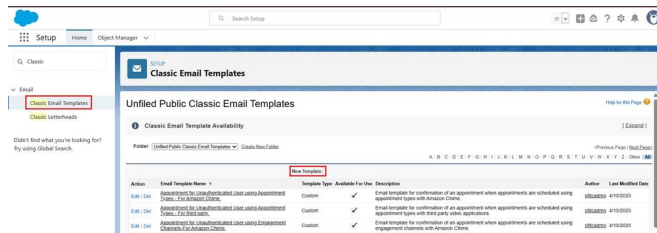
Milestone 6: Approval Process (Event Cancellation)

Complete the Profiles, Roles and Users milestones (14-16) before implementing the approval process.

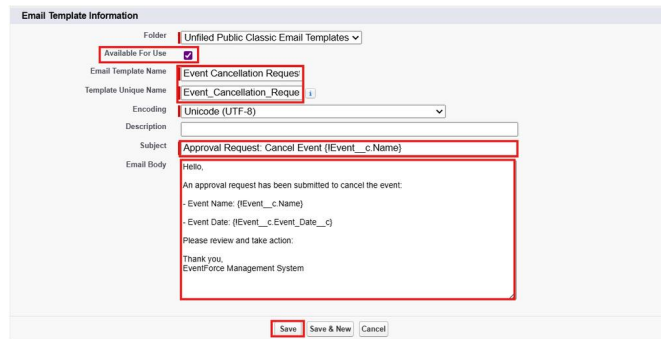
Step 1: Create four classic email templates

Setup → Quick Find **Email Templates** → **Classic Email Templates** → **New Template** → type **Text** → Next.
Folder: *Unfiled Public Email Templates*; encoding UTF-8.

Template name	Subject	Body
Event Cancellation Request Notification	Approval Request: Cancel Event {!Event__c.Name}	An approval request has been submitted to cancel the event. Event Name: {!Event__c.Name}; Event Date: {!Event__c.Event_Date__c}. Thank you, EventForce Management System
Email to Client After Cancellation	Event Cancellation Notice - {!Event__c.Name}	The event has been canceled. Event Name and Date included. If you have any questions or need assistance, please contact us. Thank you for your understanding, Event Management Team
Approval Notification to Event Owner	Event Cancellation Approved: {!Event__c.Name}	Hello {!Event__c.OwnerFullName}, your cancellation request for {!Event__c.Name} has been approved. Event Date: {!Event__c.Event_Date__c}. The event status has been updated to Canceled.
Rejection Notification to Event Owner	Event Cancellation Rejected: {!Event__c.Name}	Hello {!Event__c.OwnerFullName}, your cancellation request for {!Event__c.Name} has been rejected. Event Date: {!Event__c.Event_Date__c}. The event status has been reverted to Confirmed.



Classic Email Templates

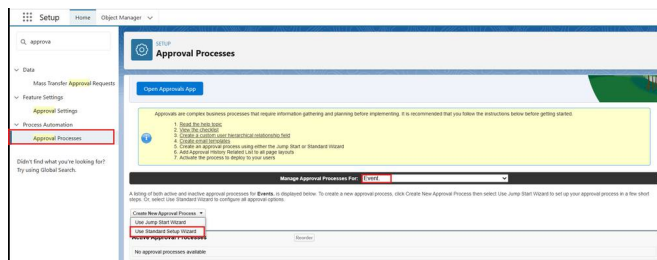


Event Cancellation Request Notification template

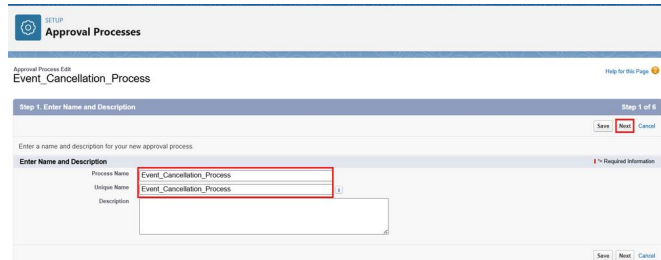
Step 2: Create the approval process

Setup → Quick Find **Approval Processes** → object **Event** → **Create New Approval Process** → Use **Standard Setup Wizard**.

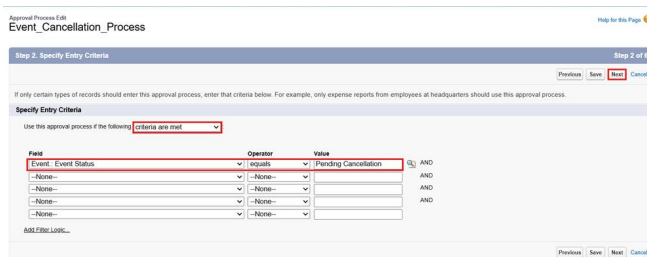
Wizard step	Configuration
1. Basic settings	Name: Event_Cancellation_Process (unique name auto-fills).
2. Entry criteria	Criteria are met: Event Status equals Pending Cancellation .
3. Approver	Automatically assign using user/queue: a specific user (Manager), or the manager of the submitting user (ensure managers are set on User records).
4. Email template	Approval assignment template: Event Cancellation Request Notification .
5. Page layout fields	Event Name, Owner, Client, Last Modified By, Created By, Event Budget, Event Date, Event Status, Event Type, Venue (reorder with Up/Down).
6. Initial submitters	Event Owner ; click Save.



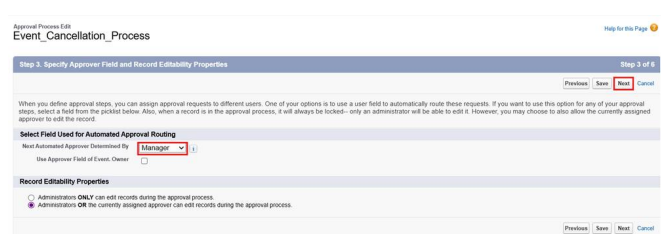
Approval Processes page



Basic settings



Entry criteria



Select approver

Approval Process Edit
Event_Cancellation_Process

Step 4 of 6

Step 4. Select Notification Templates

Select the email template that will be used to notify approvers that an approval request has been assigned to them. Note that this template will be used for all steps for this process. [Create a new email template](#)

Email Template
Approval Assignment Email Template: **Event Cancellation Request**

Previous Save Next Cancel

Approval assignment email template

Approval Process Edit
Event_Cancellation_Process

Step 6 of 6

Step 6. Select Fields to Display on Approval Page Layout

The approval page is where an approver will actually approve or reject a request. Using the options below, choose the fields to display on this page.

Available Fields: --None--

Selected Fields: Event Name, Owner, Client, Last Modified By, Created By, Event Budget, Event Date, Event Status, Event Type

Up Down

Click here to view an example

Fields shown on approval page

Approval Process Edit
Event_Cancellation_Process

Step 6 of 6

Step 6. Specify Initial Submitters

Using the options below, specify which users are allowed to submit the initial request for approval. For example, expense reports should normally be submitted for approval only by their owners.

Initial Submitters

Submitter Type: Search: Owner for Find

Available Submitters: --None--

Added Submitters: Event Owner

Add Remove

Initial submitters

Approval Processes
Event: Event_Cancellation_Process

Process Definition Detail

Process Name: Event_Cancellation_Process Active: ☒

Unique Name: Event_Cancellation_Process Next Automated Approver Determined By: Manager of Recent Submitter

Event Cancellation Request Notification

Record Editability: Administrator OR Current Approver Allow Submitters to Recall Approval Requests: ☐

Approval Assignment Email Template: Event_Cancellation_Request_Notification

Initial Submitters: Event Owner

Created By: Supantha Sath 9/23/2025, 11:12 PM Modified By: Supantha Sath 9/23/2025, 7:14 AM

Initial Submission Actions

Action	Type	Description	Time
Record Lock	Field Update	Event Status	Field Update
Record Lock	Field Update	Event Status	Field Update

Approval Steps

You have not yet defined any approval steps.

Process detail with initial actions

Step 3: Actions

Action point	Action	Settings
Initial submission	Email Alert	Description: Alert - Event Cancellation Request; Object Event__c; template Event Cancellation Request Notification; recipient Role: Event Coordinator ; From: Current User's Email Address.
Final approval	Field Update	Name: Event Status Update; field Event Status; new value Canceled .
Final approval	Email Alert	Alert - Cancellation Approved; template Approval Notification to Event Owner; recipient Event Owner; From: Current User's Email.
Final rejection	Email Alert	Alert - Cancellation Rejected; template Rejection Notification to Event Owner; recipient Event Owner.
Final rejection	Field Update	Name: Event Status To Rejected; field Event Status; new value Rejected .

Step 4: Approval step and activation

- 1 Click **Add New Step**: name **Manager Review**; criteria **All records should enter this step**.
- 2 Approver: **Automatically assign to approver(s)** → a specific user / related user (e.g. the manager user).
- 3 Click **View Approval Process Detail Page** and press **Activate**.

New Email Alert

Create an email alert to associate with one or more workflow rules, approval processes, or entitlement processes. When changing an email alert, any modifications will apply to all rules, approvals, or entitlement processes associated with it.

Email Alert Edit [Save] [Save & New] [Cancel]

Edit Email Alert

Description: Alert - Event Cancellation Request

Unique Name: Alert_Event_Cancellation_Rq

Object: Event

Email Template: Event Cancellation Request

Protected Component: ☐

Recipient Type: Role

Recipients: Role

Available Recipients:

- Role: Director, Channel Sales
- Role: Director, Direct Sales
- Role: Eastern Sales Team
- Role: Event Admin
- Role: Installation & Repair Services
- Role: Marketing Team
- Role: SVP, Customer Service & Support
- Role: SVP, Human Resources
- Role: SVP, Sales & Marketing
- Role: VP, International Sales
- Role: VP, Marketing
- Role: VP, North American Sales
- Role: Vendor Manager
- Role: Western Sales Team

Selected Recipients:

- Role: Event Coordinator

You can enter up to five (5) email addresses to be notified.

Additional Email: sushanth101@gmail.com

From Email Address: Current User's email address

[Make this address the default From email address for this object's email alerts.]

[Save] [Save & New] [Cancel]

Email alert configuration

Edit Field Update

Event Status Update

Define the field update, including the object associated with the workflow rule, approval process, or entitlement process, the field to update, and the value to apply. Note that the field to update may be on a related object. Fields are shown only for the type that you select.

Field Update Edit [Save] [Save & New] [Cancel]

Identification

Name: Event Status Update

Unique Name: Event_Status_Update

Description:

Object: Event

Field to Update: Event Status

Field Data Type: Picklist

Re-evaluate Workflow Rules after Field Change: ☐

Specify New Field Value

Picklist Options:

- ☐ The value above the current one
- ☐ The value below the current one
- ☒ A specific value: Canceled

Field update: Event Status = Canceled

Edit Email Alert

Description: Alert - Cancellation Approved

Unique Name: Alert_Cancellation_Approval

Object: Event

Email Template: Approval Notification to Event

Protected Component: ☐

Recipient Type: Role

Recipients: Role

Available Recipients:

- Role: Director, Channel Sales
- Role: Director, Direct Sales
- Role: Eastern Sales Team
- Role: Event Admin
- Role: Installation & Repair Services
- Role: Marketing Team
- Role: SVP, Customer Service & Support
- Role: SVP, Human Resources
- Role: SVP, Sales & Marketing
- Role: VP, International Sales
- Role: VP, Marketing
- Role: VP, North American Sales
- Role: Vendor Manager
- Role: Western Sales Team

Selected Recipients:

- Role: Event Coordinator

You can enter up to five (5) email addresses to be notified.

Additional Email:

From Email Address: Current User's email address

[Make this address the default From email address for this object's email alerts.]

[Save] [Save & New] [Cancel]

Rejection email alert

Field Update Edit [Save] [Save & New] [Cancel]

Identification

Name: Event Status To Rejected

Unique Name: Event_Status_To_Rejected

Description:

Object: Event

Field to Update: Event Status

Field Data Type: Picklist

Re-evaluate Workflow Rules after Field Change: ☐

Specify New Field Value

Picklist Options:

- ☐ The value above the current one
- ☐ The value below the current one
- ☒ A specific value: Rejected

[Save] [Save & New] [Cancel]

Field update: Event Status = Rejected

SETUP Approval Processes

New Approval Step [Help for this Page]

Step 1 of 3: Enter Name and Description [Next] [Cancel]

Enter a name, description, and step number for your new approval step.

Enter Name and Description

Approval Process Name: Event Cancellation Process

Unique Name: Manager_Review

Description:

Step Number: 2

[Next] [Cancel]

New approval step

Step 3: Select Assigned Approver [Previous] [Next] [Cancel]

Specify the user who should approve records that enter this step. Optionally, choose whether the approver's delegate is also allowed to approve these requests.

Select Approver

- ☐ Let the submitter choose the approver manually
- ☐ Automatically assign using the user first selected earlier (Manager)
- ☐ Automatically assign to queue
- ☒ Automatically assign to approver(s)

User: Michael Jackson

ADD Size: Normal

What happens when multiple approvers are selected:

- ☒ Approve or reject based on the FIRST response
- ☐ Require UNANIMOUS approval from all selected approvers

☐ The approver's delegate may also approve this request.

Selecting the approver

SETUP Approval Processes

Approval Processes

Event:: Event_Cancellation_Process [Help for this Page]

< Back to Approval Process List

Process Definition Detail [Edit] [Clone] [Delete] [Activate]

Process Name	Event_Cancellation_Process	Active	☐
Unique Name	Event_Cancellation_Process	Next Automated Approver Determined By	Manager of Record Submitter
Description			
Entry Criteria	Event:: Event Status EQUALS Pending Cancellation		
Record Editability	Administrator OR Current Approver		
Approval Assignment Email Template	Event Cancellation Request Notification		
Initial Submitters	Event Owner		
Created By	Sushanth Sush	9/22/2025, 11:12 PM	Modified By Sushanth Sush 9/23/2025, 9:47 AM

Approval process detail page before activation

Milestone 7: Flow - 3-Day Reminder Email

Scenario: when an Event is created/updated with Status = Confirmed and a valid Client with an email address, the system automatically emails the Client a reminder 3 days before the Event Date.

- 1 Setup → Quick Find **Flow** → **New Flow** → **Record-Triggered Flow** → Create.
- 2 Object: **Event**; trigger: **A record is created or updated**.
- 3 Condition: Event Status **Equals** Confirmed. Optimize the flow for **Actions and Related Records** → Done.
- 4 Click **Start** → **Add Scheduled Path**: label **3_Days_Before**, time source **Event__c: Event Date**, offset **3 Days Before**.
- 5 Toolbox → **New Resource** → **Text Template** named **Subject** (Plain Text): *Reminder: Your Event {!\$Record.Name} is in 3 days.*
- 6 Create a second Text Template named **Body** (Plain Text) with the text below.
- 7 Under the 3_Days_Before path click + → **Action** → **Send Email**; label **Alert_Client_3Day_Reminder**.
- 8 Recipient Address List: **Client Email** (Event → Client → Email); CC Recipient Address List: **Event Owner** email.
- 9 Email content: **Compose Email Content**, Subject = Subject resource, Body = Body resource.
- 10 Save with Flow Label **Client Reminder - 3 Days Before** and **Activate**.

Body text template

Hello {!\$Record.Client__r.Name},

This is a reminder that your event "{!\$Record.Name}" will take place on {!\$Record.Event_Date__c}.

Venue: {!\$Record.Venue__r.Name}
Type: {!\$Record.Event_Type__c}

We look forward to seeing you!

- EventForce Team

Configure Start

If you create a flow that's triggered when a record is updated, we recommend first defining entry conditions. Then select the **Only when a record is updated to meet the condition requirements** option for When to Run the Flow for Updated Records.

Condition Requirements

All Conditions Are Met (AND)

Field	Operator	Value
Event Status	Equals	Confirmed

+ Add Condition

When to Run the Flow for Updated Records

☒ Every time a record is updated and meets the condition requirements

☐ Only when a record is updated to meet the condition requirements

Optimize Flow

Optimize the Flow for:

Fast Field Updates
 Update any field on the record that triggers the flow to run. This high-performance flow runs **before the record is saved** to the database.

Actions and Related Records
 Update any record and perform actions, like send an email. This more flexible flow runs **after the record is saved** to the database.

Flow start: entry condition

Configure Scheduled Paths

SCHEDULED PATHS

Run Immediately

3_Days_Before

SCHEDULED PATH DETAILS

* Path Label: 3_Days_Before

* API Name: X3_Days_Before

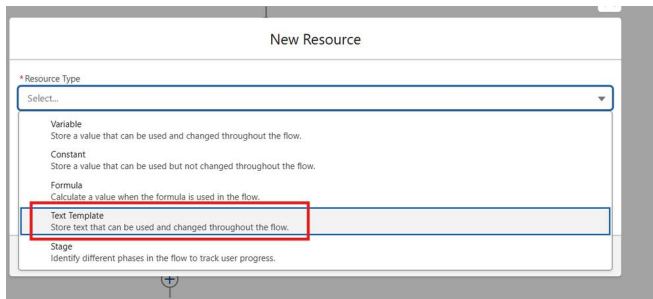
* Time Source: Event__c: Event Date

* Offset Number: 3

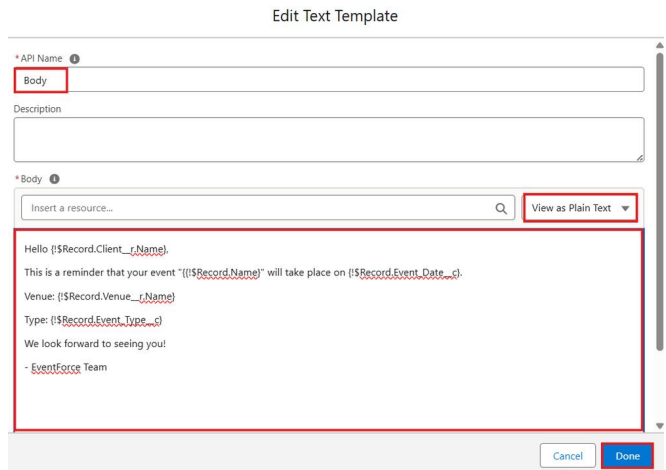
* Offset Options: Days Before

> Show advanced options

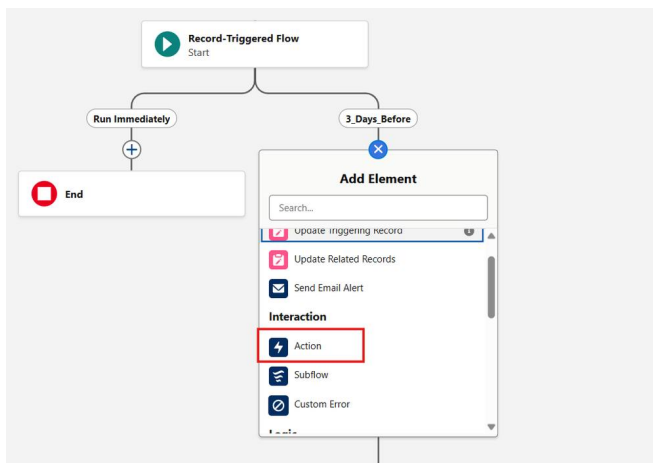
Scheduled path configuration



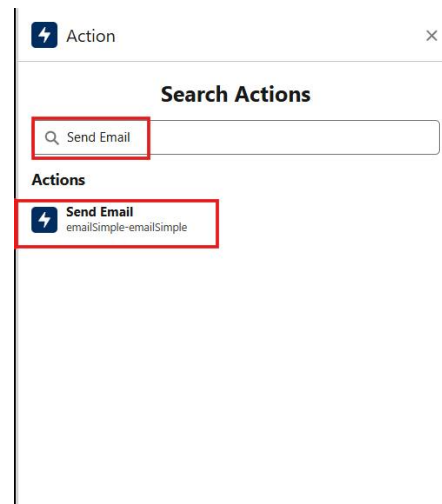
Subject text template



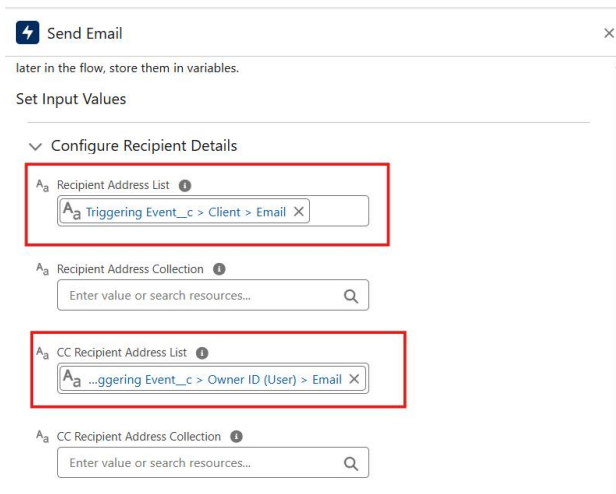
Body text template



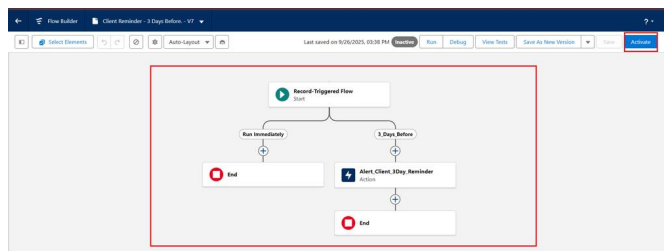
Add Action element



Search Send Email



Send Email action: recipients



Completed flow

Milestone 8: Apex Class & Trigger - Venue Status

Scenario: when an Event is created with a Venue, mark the Venue Availability as **Reserved**; when the event is cancelled, mark it **Available**.

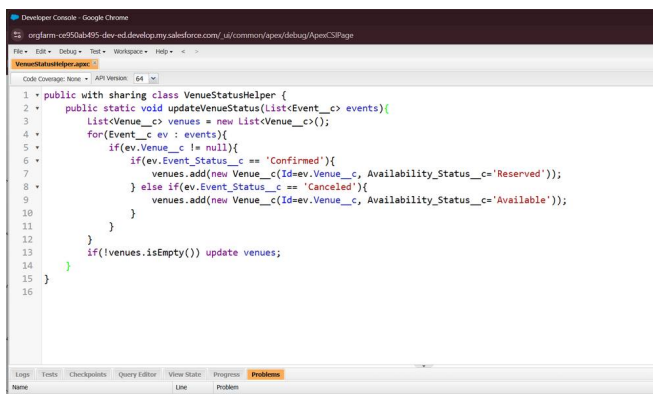
- 1 Gear icon → **Developer Console** → File → New → **Apex Class**; name **VenueStatusHelper**; paste the code and save (Ctrl+S).
- 2 File → New → **Apex Trigger**; name **EventTrigger13**, sObject **Event__c**; paste the trigger code and save.

VenueStatusHelper (Apex class)

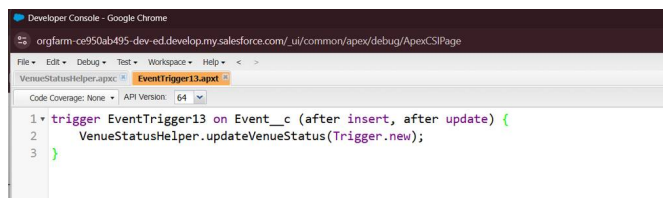
```
public with sharing class VenueStatusHelper {
    public static void updateVenueStatus(List<Event__c> events){
        List<Venue__c> venues = new List<Venue__c>();
        for(Event__c ev : events){
            if(ev.Venue__c != null){
                if(ev.Event_Status__c == 'Confirmed'){
                    venues.add(new Venue__c(Id=ev.Venue__c, Availability_Status__c='Reserved'));
                } else if(ev.Event_Status__c == 'Canceled'){
                    venues.add(new Venue__c(Id=ev.Venue__c, Availability_Status__c='Available'));
                }
            }
        }
        if(!venues.isEmpty()) update venues;
    }
}
```

EventTrigger13 (Apex trigger)

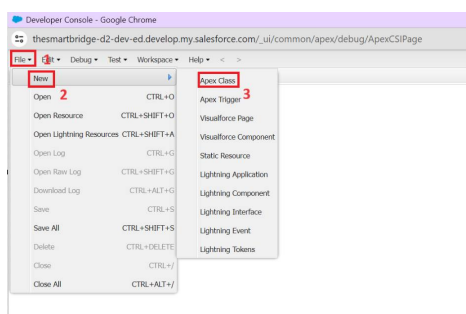
```
trigger EventTrigger13 on Event__c (after insert, after update) {
    VenueStatusHelper.updateVenueStatus(Triiger.new);
}
```



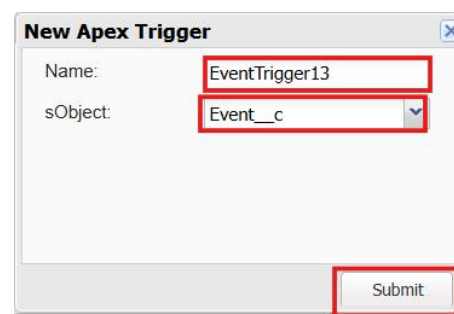
VenueStatusHelper in Developer Console



EventTrigger13 in Developer Console



New Apex class dialog



New Apex trigger dialog

Milestone 9: Apex Trigger - Prevent Double Booking

Scenario: do not allow two events at the same Venue on the same date. Create a trigger named **PreventDoubleBooking** on **Event__c** (File → New → Apex Trigger), paste the code and save.

PreventDoubleBooking (Apex trigger)

```
trigger PreventDoubleBooking on Event__c (before insert, before update) {

    Set<Id> venueIds = new Set<Id>();
    for(Event__c ev : Trigger.new){
        if(ev.Venue__c != null) venueIds.add(ev.Venue__c);
    }

    Map<Id, List<Event__c>> venueEventMap = new Map<Id, List<Event__c>>();
    for(Event__c e : [SELECT Id, Venue__c, Event_Date__c
                      FROM Event__c WHERE Venue__c IN :venueIds]){
        if(!venueEventMap.containsKey(e.Venue__c)){
            venueEventMap.put(e.Venue__c, new List<Event__c>());
        }
        venueEventMap.get(e.Venue__c).add(e);
    }

    for(Event__c ev : Trigger.new){
        if(ev.Venue__c != null && venueEventMap.containsKey(ev.Venue__c)){
            for(Event__c existing : venueEventMap.get(ev.Venue__c)){
                if(existing.Event_Date__c == ev.Event_Date__c && existing.Id != ev.Id){
                    ev.addError('This Venue is already booked on this date.');

```

Milestone 10: Batch & Schedulable Apex

Scenario: mark all past events as **Completed** every night. Create two Apex classes in the Developer Console (File → New → Apex Class).

BatchCompleteEvents (Batch Apex)

```
global class BatchCompleteEvents implements Database.Batchable<sObject> {
    global Database.QueryLocator start(Database.BatchableContext bc){
        return Database.getQueryLocator(
            'SELECT Id, Event_Date__c, Event_Status__c FROM Event__c ' +
            'WHERE Event_Date__c < TODAY AND Event_Status__c != \'Completed\''
        );
    }
    global void execute(Database.BatchableContext bc, List<Event__c> scope){
        for(Event__c ev : scope){
            ev.Event_Status__c = 'Completed';
        }
        update scope;
    }
    global void finish(Database.BatchableContext bc){
        System.debug('Past events marked as Completed');
    }
}
```

ScheduleCompleteEvents (Schedulable Apex)

```
global class ScheduleCompleteEvents implements Schedulable {
    global void execute(SchedulableContext sc){
        Database.executeBatch(new BatchCompleteEvents(), 200);
    }
}
```

Schedule the job

- 1 Setup → Quick Find **Apex Classes** → click **Schedule Apex**.
- 2 Job Name: **Daily Event Completion**; Apex Class: **ScheduleCompleteEvents**.

3 Frequency: **Weekly**, Days: **all days**, Preferred Start Time: **08:00 PM**; click **Save**.

Salesforce now runs BatchCompleteEvents every day at 8 PM and marks past events as Completed.

```
1 trigger PreventDoubleBooking on Event__c (before insert, before update) {  
2     Set<Id> venueIds = new Set<Id>();  
3     for(Event__c ev : Trigger.new){  
4         if(ev.Venue__c != null) venueIds.add(ev.Venue__c);  
5     }  
6  
7     Map<Id, List<Event__c>> venueEventMap = new Map<Id, List<Event__c>>();  
8     for(Event__c e : [SELECT Id, Venue__c, Event_Date__c  
9         FROM Event__c WHERE Venue__c IN :venueIds]){  
10         if(!venueEventMap.containsKey(e.Venue__c)){  
11             venueEventMap.put(e.Venue__c, new List<Event__c>());  
12         }  
13         venueEventMap.get(e.Venue__c).add(e);  
14     }  
15  
16     for(Event__c ev : Trigger.new){  
17         if(ev.Venue__c != null && venueEventMap.containsKey(ev.Venue__c)){  
18             for(Event__c existing : venueEventMap.get(ev.Venue__c)){  
19                 if(existing.Event_Date__c == ev.Event_Date__c && existing.Id != ev.Id){  
20                     ev.addError('This Venue is already booked on this date.');21                 }  
22             }  
23         }  
24     }  
25 }  
26 }
```

Double-booking trigger in Developer Console

The screenshot shows the 'Setup Apex Classes' page in Salesforce. The 'Job Name' is 'Daily Event Completion' and the 'Apex Class' is 'ScheduleCompleteEvents'. The 'Frequency' is set to 'Weekly', and the 'Recurs every week on' section shows all days of the week (Monday through Saturday) selected. The 'Start' date is '9/26/2025' and the 'End' date is '9/26/2025'. The 'Preferred Start Time' is '8:00 PM'. The 'Save' button is highlighted.

Schedule Apex job settings

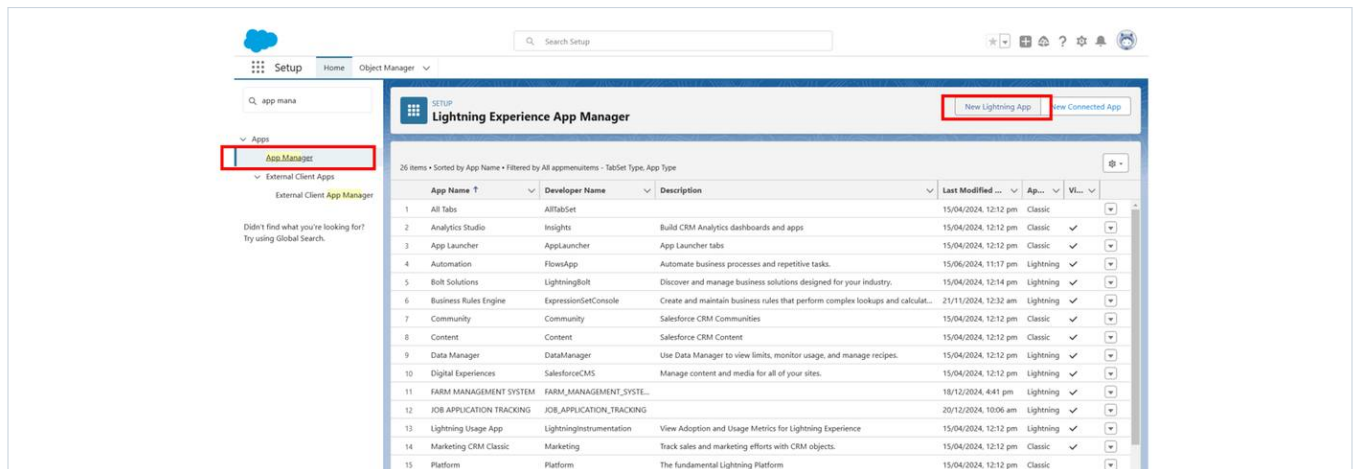
3. Phase 3: UI/UX Development & Customization

This phase creates a simple, user-friendly interface: a custom Lightning App that gathers Events, Clients, Vendors, Venues and Feedback in one workspace, plus reports and dashboards to visualize event schedules, vendor performance and client feedback.

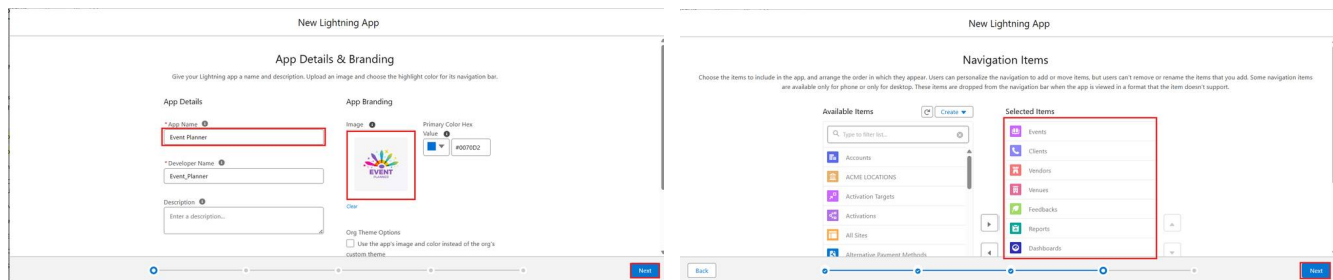
Milestone 11: Lightning App

An app is a group of tabs that work together as a unit, with a name, a logo and a set of tabs.

- 1 Setup → Quick Find **App Manager** → **New Lightning App**.
- 2 App name: **Event Planner**; upload an image; Next.
- 3 Leave App Options and Utility Items at their defaults; Next.
- 4 Move **Event, Client, Vendor, Venue, Feedback, Reports and Dashboards** to Selected Items; Next.
- 5 Move the **System Administrator** profile to Selected Profiles; **Save & Finish**.
- 6 Verify: App Launcher → search *Event Planner* → open the app.

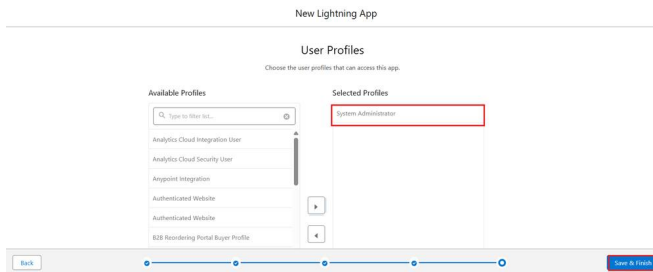


App Manager

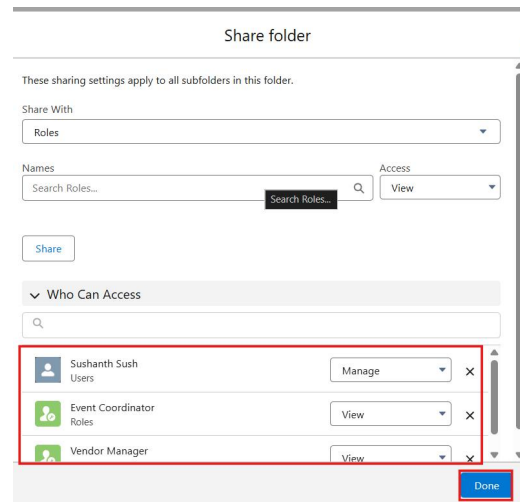


App name and branding

Navigation items



Selecting profiles for the app



Sharing the report folder

Milestone 12: Reports

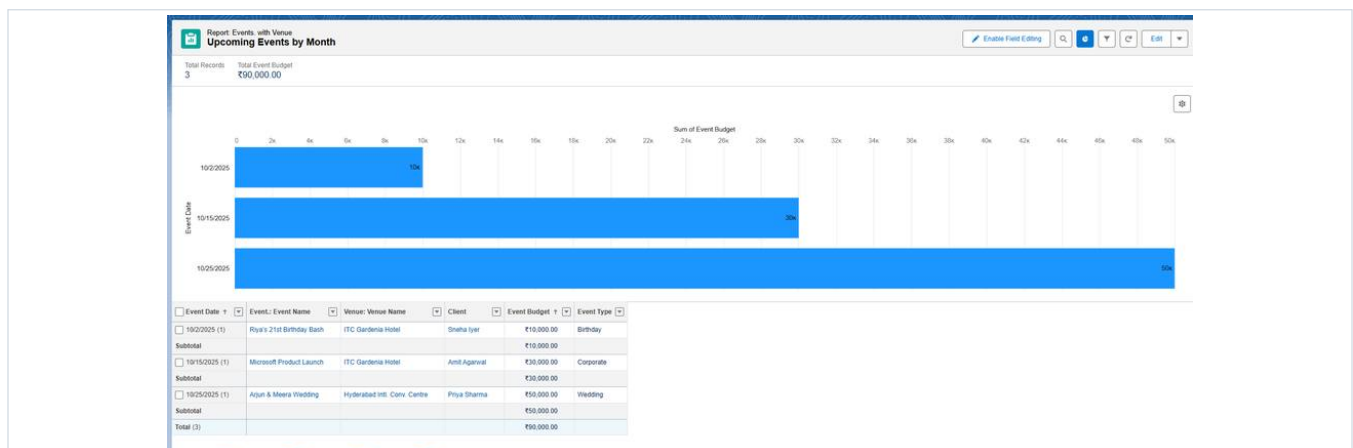
Activity 1-2: Report folder and sharing

- 1 App Launcher → **Reports** → **New Folder**; label **EventForce Report**; Save.
- 2 In **All Folders** open the folder drop-down → **Share** → share with **Roles: Event Coordinator** and **Vendor Manager** with **View** access → Share → Done.

Before creating reports, create at least 10 records in every object and fill every field for a better result.

Activity 3: Report "Upcoming Events by Month"

- 1 Reports tab → **New Report**; category **All**; select **Events** (or Events with Client & Venue) → Start Report.
- 2 Columns: Event Name, Event Date, Event Type, Client__r.Name, Venue__r.Name, Event_Budget__c; remove unnecessary fields; toggle **Update Preview Automatically**.
- 3 Filters: Show Me = All Events; Event Date ≥ TODAY (or Next 30 Days); Event Status ≠ Canceled.
- 4 Group Rows: Event Date → **Group by Calendar Month**.
- 5 Add Summary: Row Count and **SUM of Event_Budget__c** (total budget per month).
- 6 Add Chart: Column or Line; X-axis = Event Date (Calendar Month); Y-axis = Row Count or SUM(Event_Budget__c).
- 7 Save as **Upcoming Events by Month** in the **EventForce Report** folder and Run.



Upcoming Events by Month report

Milestone 13: Dashboard

- 1 In the app open the **Dashboards** tab → **New Dashboard**; name **EventForce Operations Dashboard**; Create.
- 2 Click **+ Widget** and select the **Upcoming Events by Month** report.
- 3 Chart: **Donut**; Max values displayed: **6**; Subtitle: **Upcoming Events Overview**; Widget theme: **Light**; Save.



EventForce Operations Dashboard (donut widget)

4. Phase 4: Data Migration, Testing & Security

Records for Events, Clients, Vendors, Venues and Feedback were loaded with the **Data Import Wizard** after pre-validation checks to avoid duplicates. Users were created for Event Admin, Event Coordinator, Vendor Manager and Client, each with a profile and role matching their access level. Organization-Wide Defaults were set to **Private**, with controlled access through sharing rules such as *Event_Sharing_For_Vendors*. Testing validated data accuracy, automation performance and user permissions.

Milestone 14: Profiles

A profile is a collection of settings and permissions that defines what a user can do: object, field and user permissions, tab and app settings, Apex class access, page layouts, record types, login hours and IP ranges. A profile can be assigned to many users, but a user has only one profile.

Activity 1: Event Admin

Setup → Profiles → **New Profile** → existing profile **System Administrator**; name **Event Admin**; Save. System Administrator is the base because it already has Modify All Data, Customize Application and full CRUD on all objects, so Event Admin inherits full access (restrict later if needed).

Activities 2-4: Event Coordinator, Vendor Manager and Client Profile

For each: Profiles → **New Profile** → clone **Standard Platform User** → enter the name → Save → **Edit**. Set the Custom App default to **Event Planner**, set the object permissions below, session timeout **2 hours of inactivity**, password expiry **never expires**, minimum password length **8**, then Save.

Object	Event Coordinator	Vendor Manager	Client Profile
Event	Create, Read, Edit, Delete	Read only	Read only (own events)
Client	Read, Edit	Read only	-
Vendor	Read	Create, Read, Edit, Delete	No access
Venue	Read	Read only	No access
Feedback	Create, Read	Read only	Create, Read (give feedback)

Clone Profile

Enter the name of the new profile.

You must select an existing profile to clone from.

Existing Profile: **System Administrator**

User License: **Salesforce**

Profile Name: **Event Admin**

Save **Cancel**

Cloning a profile

Event Coordinator

Users with this profile have the permissions and page layouts listed below. Administrators can change a user's profile by editing that user's personal information.

If your organization uses Record Types, use the Edit links in the Record Type Settings section below to make one or more record types available to users with this profile.

Look Up Roles: [Enabled Apex Class Access \(0\)](#) | [Enabled Visualforce Page Access \(0\)](#) | [Enabled External Data Source Access \(0\)](#) | [Enabled Named Credential Access \(0\)](#) | [Enabled External Connected Platform Access \(0\)](#) | [Enabled Custom Metadata Type Access \(0\)](#) | [Enabled Custom Setting Definition Access \(0\)](#) | [Enabled Flow Access \(0\)](#) | [Enabled Service Provider Status Access \(0\)](#) | [Enabled Custom Permissions \(0\)](#)

Profile Detail

Name: **Event Coordinator** **Edit** **Close** **Delete** **View Users**

User License: **Salesforce Platform**

Description: **Custom Profile** ✓

Created By: **Subhath Subh** 9/14/2025, 11:49 PM Modified By: **Subhath Subh** 9/14/2025, 11:49 PM

Event Coordinator profile

Custom Object Permissions

	Read	Create	Edit	Delete	View All Records	Modify All Fields
ACME LOCATIONS	☐	☐	☐	☐	☐	☐
Clients	☒	☒	☒	☐	☐	☐
Events	☒	☒	☒	☐	☐	☐
Event Vendors	☐	☐	☐	☐	☐	☐

Event Coordinator object permissions

Custom Object Permissions

	Read	Create	Edit	Delete	View All Records	Modify All Fields
ACME LOCATIONS	☐	☐	☐	☐	☐	☐
Clients	☐	☐	☐	☐	☐	☐
Events	☒	☐	☐	☐	☐	☐
Event Vendors	☐	☐	☐	☐	☐	☐

Client profile object permissions

Session Settings

Session Times Out After: **2 hours of inactivity**

Session Security Level Required at Login: **None**

Password Policies

User passwords expire in: **Never expires**

Enforce password history: **3 passwords remembered**

Minimum password length: **8**

Password complexity requirement: **Must include alpha and numeric characters**

Password question requirement: **Cannot contain password**

Maximum invalid login attempts: **10**

Lockout effective period: **15 minutes**

Obscure secret answer for password resets: ☐

Require a minimum 1 day password lifetime: ☐

Don't immediately expire links to longer password emails: ☐

Save **Save & New** **Cancel**

Session and password policies

Profiles

All Profiles **Edit** **Delete** **Create New View**

New Profile **+**

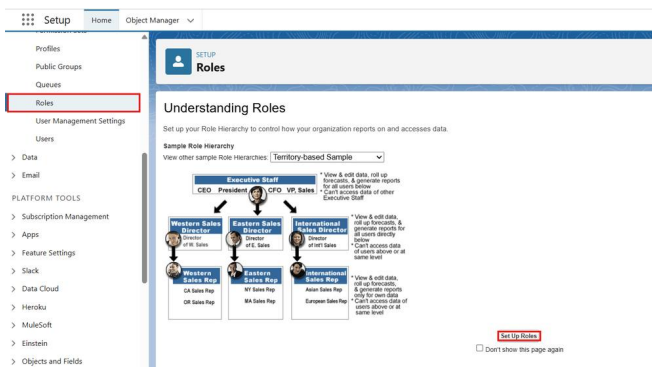
Action	Profile Name	User License
☐ Edit Clone	Analytics Cloud Integration User	Analytics Cloud Integration User
☐ Edit Clone	Analytics Cloud Security User	Analytics Cloud Integration User
☐ Edit Clone	Anypoint Integration	Identity

Profile list

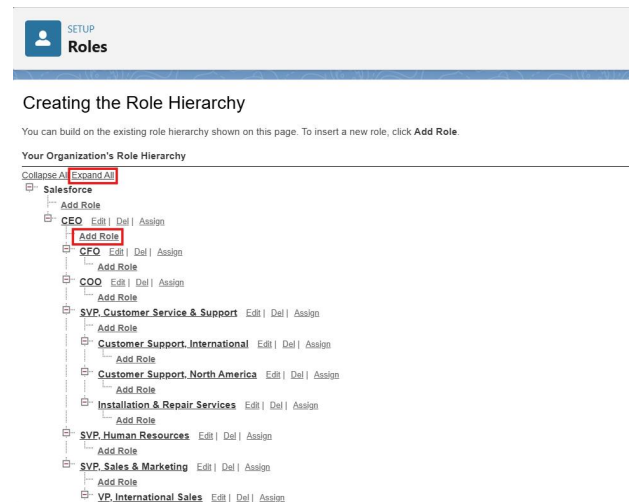
Milestone 15: Roles

Roles control record visibility and define the reporting hierarchy; users in higher roles see more data.

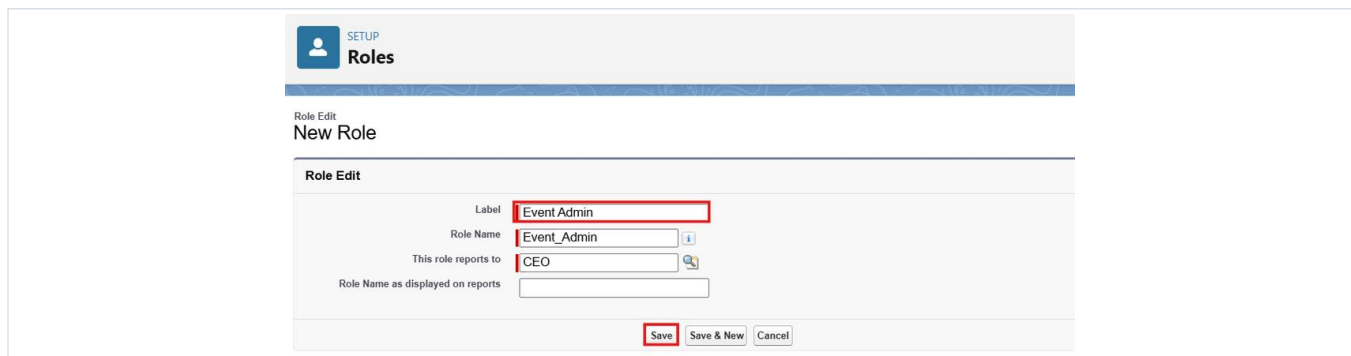
- 1 Quick Find **Roles** → **Set Up Roles** → **Expand All** → **Add Role** under CEO.
- 2 Label **Event Admin** (role name auto-populates) → Save.
- 3 Under Event Admin add three child roles the same way: **Event Coordinator**, **Vendor Manager** and **Client**.



Understanding roles



Role hierarchy



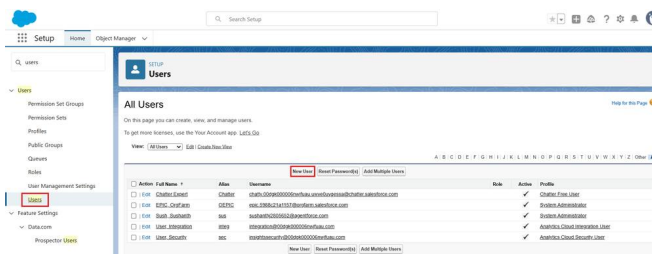
Creating the Event Admin role

Milestone 16: Users

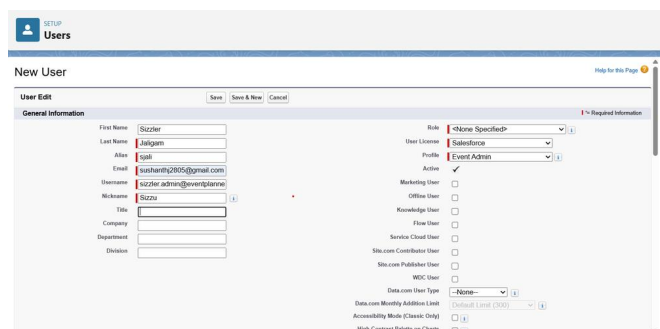
A user is anyone who logs in to Salesforce; the user account settings determine which features and records they can access.

A Developer Org allows only 2 Salesforce-license users at a time. If no Salesforce license is available, deactivate a user holding one or change the license type.

- 1 Setup → Quick Find **Users** → **New User**.
- 2 Admin user: First name **Sizzler**, Last name **Jaligam**, alias, personal email, unique username (format text@text.text), nickname, Role **Event Admin**, User License **Salesforce**, Profile **Event Admin**; Save.
- 3 Repeat for the other users with Role/Profile = **Event Coordinator**, **Vendor Manager** and **Client**, using User License **Salesforce Platform**.



All Users list



New User form

Milestone 17: Permission Set

A permission set grants extra permissions without changing a user's profile.

- 1 Setup → Quick Find **Permission Set** → **New**; label **Feedback Manager**; Save.
- 2 Open **Object Settings** → **Feedback** → Edit → enable **Read, Edit, Create, Delete** → Save.
- 3 **Manage Assignments** → **Add Assignment** → select the Event Coordinator role user → **Next** → **Assign**.

Permission Set: Feedback Manager

Find Settings... Clone Delete Edit Properties Manage Assignments View Summary

Permission Set Overview > Object Settings > Feedbacks

Feedbacks Save Cancel

Tab Settings

Available	Visible
☐	☐

Object Permissions

Permission Name	Enabled
Read	☒
Create	☒
Edit	☒
Delete	☒
View All Records	☐
Modify All Records	☐
View All Fields	☐

Feedback object permissions

Feedback Manager

Select Users to Assign

All Users

1 item selected

Full Name	Alias	Username	Role	Active	Profile
Ananya Pandey	anand	ananya.client@eventplanner.com	Client	☒	Client Profile
Chatter Expert	Chatter	chatter.003gk00000mgjauuawebungressa@chatter.salesforce.com		☒	Chatter Free User
Integration User	Integ	integration@003gk00000mgjauuawebungressa@chatter.salesforce.com		☒	Analytics Cloud Integration User
Michael Jackson	mjack	michael.coordinator@eventplanner.com	Event Coordinator	☒	Event Coordinator
Orglarm EPC	OEPC	epic.598Bz2ta1157@orglarm.salesforce.com		☒	System Administrator

Cancel Assign

Assigning the permission set

Milestone 18: Sharing Settings

Sharing settings control how records are accessed and shared. **Organization-Wide Defaults (OWD)** set the baseline access to each object; role hierarchy, sharing rules, manual sharing and Apex sharing can only *extend* access beyond it.

OWD option	Meaning
Private	Only the owner and users above them in the role hierarchy can view or edit.
Public Read Only	All users can view; only the owner and users above can edit.
Public Read/Write	All users can view and edit regardless of ownership.
Controlled by Parent	Child record access follows the parent record's sharing settings.

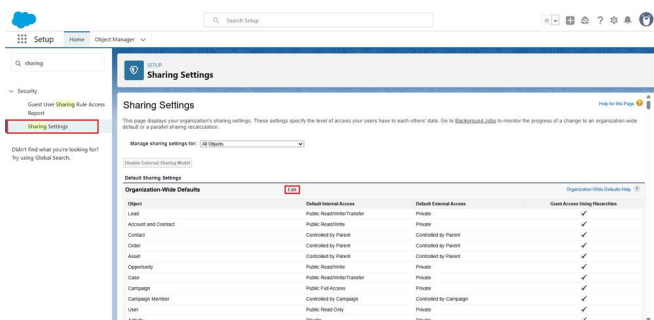
Activity 1: Configure OWD

Setup → **Sharing Settings** → Edit → under Organization-Wide Defaults select the **Event** object → Edit → Default Internal Access = **Private** and Default External Access = **Private** → Save.

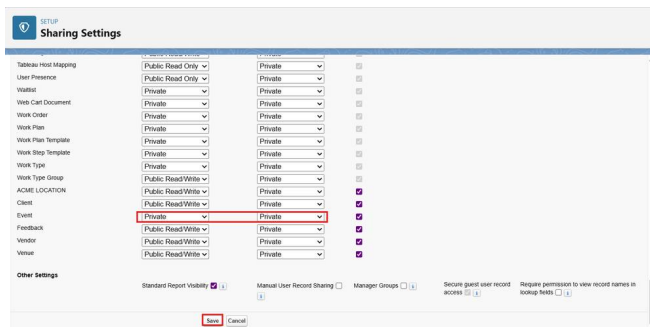
Sharing rule: Event_Sharing_For_Vendors

Sharing rules extend access for objects whose OWD is Private or Public Read Only; they can be owner-based or criteria-based.

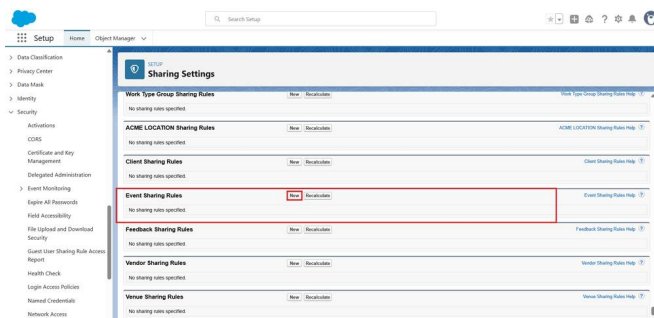
- 1 Sharing Settings → scroll to the **Event** sharing rules → **New**.
- 2 Label **Event_Sharing_For_Vendors** (rule name auto-populates).
- 3 Records to share: members of **Roles** → **Event Coordinator**.
- 4 Share with: **Roles** → **Vendor Manager**; access level **Read Only**; Save.



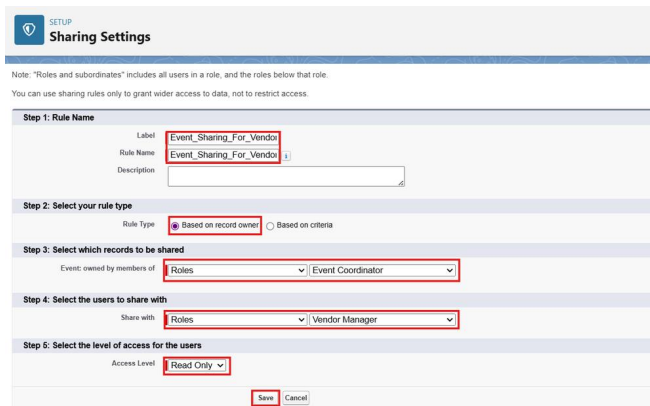
Organization-wide defaults



Event object set to Private



Sharing rules list



Sharing rule configuration

5. Phase 5: Deployment, Documentation & Maintenance

Deployment

- All configuration, customization and automation was built and tested in a **Salesforce Developer Edition Org**.
- The aim is to understand how a real-world Salesforce CRM solution is prepared for deployment after configuration, testing and security setup.
- A production deployment is not performed because Developer Edition orgs are standalone and not connected to production instances.
- Real deployments use sandboxes, change sets or DevOps tools (Salesforce DX, GitHub CI/CD pipelines).
- This phase therefore simulates deployment: objects, fields, flows, Apex and dashboards are structured, version-controlled and ready for migration.

Maintenance, Monitoring & Troubleshooting

Maintenance keeps event, client, vendor and feedback data accurate. Regular monitoring of flows, Apex jobs and approval processes surfaces failed automations early. Troubleshooting covers:

- Resolving user-access errors and permission conflicts.
- Debugging Apex triggers and flow logic.
- Adjusting validation rules or sharing settings as business needs evolve.

Project Documentation

Documentation records the project's purpose, data model, configuration, automation and testing, mapping business requirements to Salesforce features and supporting scalability, training and knowledge transfer. It covers:

- Business overview and objectives.
- Phase-wise implementation (Requirement → Development → UI → Testing → Security).
- Data model and ER diagram.
- Automation components: flows, approval process, validation rules and Apex logic.
- Security model: profiles, roles, permission sets and sharing rules.
- Testing results and sample screenshots of all Salesforce features.

Conclusion

The EventForce Management System shows how Salesforce can streamline and automate event-planning operations through customized objects, automation and analytics. Although built in a Developer Org for academic purposes, the project reflects real-world CRM implementation practice, combining a strong data model, automation logic and user-experience design. The system is fully tested, documented and ready for enterprise-level deployment and maintenance.